

Knowledge Execution Architecture

Upload !' Approval !' Ingestion !' Chunking !' Embedding !' Hybrid Retrieval
!' Evidence Pack !' Runtime Manifest !' Specialist !' Completed Work !' Audit Trail

NeedsOps AI+ · Sprint 27.3 · Internal Architecture Reference

Pipeline Overview



End-to-End Data Lineage

Tracing a single Medication Administration Policy through every database table:

knowledge_sources	id="src-medpol-001"	Parent record; status!'approved
knowledge_source_versions	id="ver-medpol-v3"	isCurrent=true; ingestionStatus!'complete
ingestion_jobs	id="job-abc123"	status: queued!'processing!'complete
knowledge_chunks	chunk-001 ... chunk-047	chunkIndex, sectionTitle, pageNumber, embedding
retrieval_audit_events	executionId="exec-xyz"	scores, chunkIds, selectionReasons
work_package_manifests	id="wpm-456"	Immutable; specialist + blueprintId + sourceRefs
completed_work	id="cw-789"	status: draft!'awaiting_approval!'approved
completed_work_assets	citationRef="Policy,v3,\$4"	Permanent citation trail per source chunk
org_audit_log	event=completed_work.approved	Actor, timestamp; never aborts business op

Key Invariants		
Only approved, current sources appear in retrieval		hybridRetrievalService SQL hard filter
Approval status preserved across version replacements		replaceSourceVersion — status excluded from UPDATE SET
Each manifest is immutable		work_package_manifests is append-only; no UPDATE path
Chunks below 0.05 confidence discarded		resolveEvidence confidence gate
Audit writes never abort business operations		logOrgEvent wraps every write in .catch(()=>{})
Tenant isolation enforced at every DB query		organizationId in all predicates + Postgres RLS (70 tables)
LLM blueprint classification only when keyword conf=0		selectBlueprint two-stage gate in workBlueprintService.ts

Stage 1 — Upload

Admin submits file !' source row created with status=uploaded

ENTRY POINT POST /v1/organisations/:orgSlug/library/sources

SERVICE knowledgeSourceService.ts

KEY FUNCTIONS

completeUpload(input)

findDuplicateChecksum(checksum, orgId)

DATABASE TABLES

TABLE	OP	KEY COLUMNS / DETAILS
knowledge_sources	INSERT	id, organizationId, title, sourceType, authorityLevel, sensitivityClassification, status="uploaded"
knowledge_source_versions	INSERT	id, knowledgeSourceId, versionLabel, storageKey, checksum, fileSize, mimeType, ingestionStatus="pending"

GUARDS & VALIDATION

%, Actor must hold library:write permission on the organisation

%, sourceType must be a member of the allowed enum (policy, procedure, legislation ...)

%, Duplicate checksum within same org !' 409 idempotent (existing version returned)

%, Empty or whitespace-only title rejected before insert

EMITS knowledge.source.uploaded !' logOrgEvent (best-effort, non-blocking)

NOTE

GCS signed URLs cannot be issued from the Replit sidecar. All uploads are proxied through the API server using file.save(buffer) with service-account credentials. Never call GCS directly from client code.

Stage 2 — Approval

Reviewer approves !' source becomes visible to every specialist

ENTRY POINT POST /v1/organisations/:orgSlug/library/sources/:sourceId/approve

SERVICE knowledgeSourceService.ts

KEY FUNCTIONS

approveKnowledgeSource(sourceId, orgId, actorUserId)

revokeKnowledgeSource(...)

supersedeKnowledgeSource(...)

replaceSourceVersion(...)

DATABASE TABLES

TABLE	OP	KEY COLUMNS / DETAILS
knowledge_sources	UPDATE	status = "approved"
knowledge_source_versions	UPDATE	status = "approved" on active version
knowledge_source_scopes	INSERT	scope assignment if provided with approval

GUARDS & VALIDATION

%, hybridRetrievalService hard-filters: status="approved" AND is_current=true AND deleted_at IS NULL

%, replaceSourceVersion — status field intentionally absent from UPDATE SET

%, Superseded sources set status="superseded"; history preserved in retrieval_audit_events

%, Revoked sources: revokedAt timestamp set; immediate removal from retrieval

EMITS knowledge.source.approved / revoked / superseded !' logOrgEvent

NOTE

Approval is the single most critical gate. An uploaded-but-unapproved document is completely invisible to specialists regardless of ingestion state. The only way to make a document searchable is explicit approval by an authorised actor.

Stage 3 — Ingestion

Background worker claims job atomically !' extracts text from file

ENTRY POINT Internal queue worker; auto-enqueued by completeUpload and replaceSourceVersion

SERVICE DatabaseIngestionQueue.ts + knowledgeOrchestrationEngine.ts

KEY FUNCTIONS

DatabaseIngestionQueue.claimNext()
DatabaseIngestionQueue.heartbeat(jobId)
knowledgeOrchestrationEngine.process(job)

DATABASE TABLES

TABLE	OP	KEY COLUMNS / DETAILS
ingestion_jobs	UPDATE	status: queued!'processing!'complete (or failed/dead_lettered); attemptCount++
knowledge_source_versions	UPDATE	ingestionStatus, ingestionMetadata (page count, error message)

GUARDS & VALIDATION

- %, SELECT ... FOR UPDATE SKIP LOCKED — atomic claim; multiple workers cannot double-process the same job
- %, Heartbeat updates prevent stale leases from blocking the queue indefinitely
- %, attemptCount "e maxAttempts !" dead_lettered; operator intervention required
- %, Zero chunks produced (corrupt/empty file) !" failed with error in ingestionMetadata

EMITS n/a (internal state machine transitions only)

NOTE
replaceSourceVersion uses the pre-computed newVersionId UUID (assigned at function entry with randomUUID()) in the fire-and-forget enqueue call — not newVersion!.id — to avoid a race between the post-transaction SELECT and the enqueue call.

Stage 4 — Chunking

Extracted text split into token-bounded, position-annotated chunks

ENTRY POINT Internal – called by orchestration engine immediately after text extraction

SERVICE knowledgeOrchestrationEngine.ts (inside ingestion worker)

KEY FUNCTIONS

chunkText(extractedText, options)
insertChunks(chunks, versionId, orgId)

DATABASE TABLES

TABLE	OP	KEY COLUMNS / DETAILS
knowledge_chunks	INSERT	id, organizationId, knowledgeSourceId, sourceVersionId, chunkIndex, sectionTitle, headingPath, pageNumber, text, tokenCount, contentHash

GUARDS & VALIDATION

- %_ Token budget per chunk ~512 tokens (configurable per blueprint)
- %_ contentHash (SHA-256 of chunk text) enables deduplication on re-ingestion
- %_ PDF parser preserves section headings + page numbers as chunk metadata
- %_ Zero chunks produced != job transitions to failed status

EMITS n/a

NOTE
Chunk metadata (sectionTitle, headingPath, pageNumber) is the source of citation strings like "Section 4, p.†12" shown in the evidence pack and stored permanently in completed_work_assets.citationRef. Without this metadata citations would be opaque byte offsets.

Stage 5 — Embedding

Each chunk receives a 1536-dim pgvector + implicit tsvector for full-text search

ENTRY POINT Internal – called immediately after chunking within the same ingestion job

SERVICE knowledgeOrchestrationEngine.ts + knowledgeStorageService.ts

KEY FUNCTIONS

embedChunks(chunks)

updateChunkEmbeddings(chunkIds, embeddings)

DATABASE TABLES

TABLE	OP	KEY COLUMNS / DETAILS
knowledge_chunks	UPDATE	embedding vector(1536), embeddingModel, tsvector GIN-indexed (implicit)

GUARDS & VALIDATION

%, AI_PROVIDER=openai !' OpenAI text-embedding-3-small via AI gateway proxy

%, Provider unavailable !' job fails and retries; partial chunk states are never committed

%, Chunks without embeddings receive semanticScore=0.0 at retrieval time (not excluded)

%, Current pipeline passes queryEmbedding:null — lexical-only mode; semantic activates when provided

EMITS n/a

NOTE

The pipeline currently runs in lexical-only mode because queryEmbedding is passed as null in resolveEvidence. Semantic scoring (cosine similarity) is fully wired in hybridRetrievalService and activates automatically the moment a query embedding is supplied.

Stage 6 — Hybrid Retrieval

Semantic cosine + BM25-style lexical scores combined; freshness + authority bonuses applied

ENTRY POINT retrieveChunks(input) – called from knowledgeResolutionService.resolveEvidence

SERVICE hybridRetrievalService.ts

KEY FUNCTIONS

retrieveChunks(input)
computeFreshnessBonus(effectiveFrom)
computeAuthorityBonus(authorityLevel)

DATABASE TABLES

TABLE	OP	KEY COLUMNS / DETAILS
knowledge_chunks (kc)	SELECT JOIN	text, embedding, chunkIndex, sectionTitle, pageNumber, tokenCount
knowledge_sources (ks)	JOIN	status, is_current, authorityLevel, sensitivityClassification, effectiveDates
retrieval_audit_events	INSERT	executionId, queryText, chunkIds[], scores{}, selectionReasons, metrics

GUARDS & VALIDATION

% , ks.organization_id = \$orgId — tenant isolation; never crosses org boundary
% , ks.status = "approved" — unapproved sources are invisible to this query
% , ks.is_current = true — superseded versions are excluded from results
% , kc.deleted_at IS NULL — soft-deleted chunks excluded
% , Effective date range: effective_from "d today "d effective_to (when set on source)
% , sensitivity_classification matched against specialist clearance level

EMITS retrieval_audit_events row written per retrieveChunks call (permanent forensic record)

NOTE
Scoring formula: baseScore = (semanticScore × 0.6) + (lexicalScore × 0.4) + freshnessBonus + authorityBonus. Weights are configurable per blueprint.
The retrieval_audit_events row records every score component per chunk, enabling full reproducibility.

Stage 7 — Evidence Pack

Chunks assembled, confidence-filtered, enriched, cached, and formatted as prompt section

ENTRY POINT resolveEvidence(input) != buildEvidenceSection(pack) != buildCitationSummary(pack)

SERVICE knowledgeResolutionService.ts

KEY FUNCTIONS

resolveEvidence(input)
buildEvidenceSection(pack)
buildCitationSummary(pack)
invalidateEvidenceCache(executionId)

DATABASE TABLES

TABLE	OP	KEY COLUMNS / DETAILS
knowledge_chunks	SELECT	Direct sourceId query for task-upload path (not via hybrid retrieval)
knowledge_source_versions	SELECT .limit(500)	Batch version-label enrichment
knowledge_sources	SELECT .limit(200)	Batch source-type lookup for citationsByType grouping

GUARDS & VALIDATION

%_Confidence gate: chunks with baseScore < 0.05 are discarded before packing
%_In-process cache keyed on executionId — second call for same execution returns instantly
%_Task-upload chunks fetched via direct sourceId query (no hybrid retrieval path)
%_All batch enrichment queries use .limit() safety caps (500 / 200 / 100)

EMITS EvidencePack (in-memory); buildCitationSummary != JSON array for completed_work_assets

NOTE buildEvidenceSection produces the "=== AUTHORITATIVE EVIDENCE ===" block injected into the specialist prompt, grouped by source type (Legislation / Policy / Procedure ...). buildCitationSummary produces the JSON stored in completed_work_assets so the citation trail survives outside the system.

Stage 8 — Runtime Manifest

Immutable snapshot of all inputs bound to this execution — written once, never updated

ENTRY POINT assembleWorkPackage(input) – called by workExecutionPipelineService before LLM call

SERVICE workPackageService.ts + workBlueprintService.ts

KEY FUNCTIONS

assembleWorkPackage(input)
selectBlueprint(userRequest, orgId)
classifyBlueprintWithLLM(userRequest, orgId)
validateWorkPackage(pkg)

DATABASE TABLES

TABLE	OP	KEY COLUMNS / DETAILS
work_package_manifests	INSERT	id, executionId, orgId, primarySpecialist, blueprintId/Version, librarySources, taskUploads, cosMemories, specialistMemories, entityKnowledge, assembledAt

GUARDS & VALIDATION

%_ Blueprint selection: keyword fast path first; LLM fallback only when keyword confidence = 0
%_ LLM classification requires confidence "e 0.6 — weak matches are rejected (no guess routing)
%_ Org-published blueprints override built-in blueprints for the same code
%_ validateWorkPackage: missing required inputs !' awaiting_clarification (not a degraded run)
%_ Manifest is append-only — no UPDATE path exists anywhere in the codebase

EMITS n/a (written synchronously before the LLM call begins)

NOTE
The manifest is the source of truth for audit, reproducibility, and cost attribution. Every evidence citation, memory entry, and blueprint version used is recorded here. The LLM call cannot begin until the manifest row is committed.

Stage 9 — Specialist

LLM called with Employee File constitution + evidence section; self-review scores draft

ENTRY POINT executeWork(input) !' generateDraft(pkg, evidence) !' reviewDraft(draft)

SERVICE workExecutionPipelineService.ts + specialistContextService.ts

KEY FUNCTIONS

executeWork(input)
generateDraft(workPackage, evidencePack)
reviewDraft(draft, criteria)
loadSpecialistContext(specialistId, orgId)
buildSpecialistContext(context)

DATABASE TABLES

TABLE	OP	KEY COLUMNS / DETAILS
completed_work	INSERT via createDraft	id, organizationId, specialistId, executionId, status="draft"

GUARDS & VALIDATION

% Evidence retrieval failure is non-fatal — metadata-only fallback + warning label in prompt
% Token budget overrun !' memory entries truncated oldest-first (most-recent preserved)
% Draft generation failure !' execution_failed status + SSE notification to connected client
% Self-review: 10 quality dimensions, each scored 0–10; stored alongside draft

EMITS SSE events: task_auto_created (if CoS confidence "e 0.85 + shouldCreateTask), done / error

NOTE
The specialist system prompt comes entirely from its Employee File (constitution, personality, prohibited phrases, output format). The evidence pack is injected as the "=== AUTHORITATIVE EVIDENCE ===" section. Self-review runs immediately after generation before the record is stored.

Stage 10 — Completed Work

Draft stored; approval lifecycle managed; permanent citation trail written

ENTRY POINT createDraft(input) / submitForApproval(id) / approveWork(id) / rejectWork(id)

SERVICE completedWorkService.ts

KEY FUNCTIONS

```
createDraft(input)
submitForApproval(id, actorId)
approveWork(id, actorId)
rejectWork(id, actorId, reason)
exportAsPdf(id)
exportAsDocx(id)
```

DATABASE TABLES

TABLE	OP	KEY COLUMNS / DETAILS
completed_work	INSERT/UPDATE	id, status machine (draft!'awaiting_approval!'approved/rejected), specialistId, manifestRef
completed_work_versions	INSERT	Immutable content snapshot per approval cycle
completed_work_assets	INSERT	One row per source citation; citationRef = "Policy, v3, §4, p.12"
completed_work_comments	INSERT	Reviewer comments threaded per version

GUARDS & VALIDATION

%_ Status machine enforced: only valid transitions accepted (no arbitrary status jumps)

%_ rejected !' draft (rework) path re-enters the pipeline for revision

%_ approved !' archived preserves all version + asset + citation history

%_ citationRef persists the human-readable citation outside the source document

EMITS completed_work.submitted / approved / rejected !' logOrgEvent

NOTE

completed_work_assets is the permanent citation trail. Every unique sourceId from the EvidencePack produces one row with a citationRef string. This survives export to PDF/DOCX, ensuring evidence attribution is preserved when work leaves the system.

Stage 11 — Audit Trail

Every transition recorded; retrieval query persisted with full per-chunk scoring detail

ENTRY POINT logOrgEvent(event) – called at every significant transition across all 10 prior stages

SERVICE auditService.ts + knowledgeOrchestrationEngine.ts

KEY FUNCTIONS

logOrgEvent(event)
writeAuditEvent(event)
log(event)

DATABASE TABLES

TABLE	OP	KEY COLUMNS / DETAILS
org_audit_log	INSERT	Per-org events: upload, approve, revoke, execute, complete, approve — actor + timestamp
platform_audit_log	INSERT	Platform-scoped: org provisioning, plan changes, staff actions
retrieval_audit_events	INSERT	Per-execution: queryText, chunkIds[], scores{semantic,lexical,base,fresh,auth}, selectionReasons, metrics

GUARDS & VALIDATION

- %, logOrgEvent wraps every insert in .catch(()=>{}) — audit NEVER aborts a business operation
- %, Falls back to public org_audit_log schema if tenant schema not yet provisioned
- %, retrieval_audit_events: one row per retrieveChunks call (not per individual chunk)
- %, RLS: REQUIRED_RLS_TABLES=70 verified at every server startup; startup fails if count drops

EMITS n/a (terminal stage)

NOTE
The retrieval_audit_events row is the forensic record for explaining any specialist output. It contains the exact query, which chunks were shown, and every score component — enabling full reproducibility and compliance review of any piece of completed work.

Service & Schema File Index

STAGE	PRIMARY FILE(S)
Upload	artifacts/api-server/src/services/knowledgeSourceService.ts
Approval	artifacts/api-server/src/services/knowledgeSourceService.ts
Ingestion	artifacts/api-server/src/lib/ingestionQueue/DatabaseIngestionQueue.ts
Chunking	artifacts/api-server/src/lib/knowledgeOrchestrationEngine.ts
Embedding	artifacts/api-server/src/lib/knowledgeOrchestrationEngine.ts
Hybrid Retrieval	artifacts/api-server/src/services/hybridRetrievalService.ts
Evidence Pack	artifacts/api-server/src/services/knowledgeResolutionService.ts
Runtime Manifest	artifacts/api-server/src/services/workPackageService.ts + workBlueprintService.ts
Specialist	artifacts/api-server/src/services/workExecutionPipelineService.ts + specialistContextService.ts
Completed Work	artifacts/api-server/src/services/completedWorkService.ts
Audit Trail	artifacts/api-server/src/services/auditService.ts

DB Schema Files (lib/db/src/schema/)

knowledgeSources.ts

knowledgeSourceVersions.ts

knowledgeSourceScopes.ts

knowledgeChunks.ts

ingestionJobs.ts

retrievalAuditEvents.ts

workPackageManifests.ts

completedWork.ts

completedWorkVersions.ts

completedWorkAssets.ts

orgAuditLog.ts

RLS table count verified at API server startup: REQUIRED_RLS_TABLES = 70 · lib/org-db/src/rlsVerifier.ts

